

Information Theory Final Project

Range Coding and a Simplified DEFLATE Compressor

Team members

Name	ID
Omar Hani Bishr	21010891
Mohamed Mohamed Ibrahim Ali Hassan	21011211
Mohamed Hassan Mohamed Qabari	21011115
Ahmed Samir El-sayed	21010094
Nayra Ibrahim Ahmad Mohamed	21011504

Code Repo: [link](#)

Test cases: [link](#)

1. A short explanation of range coding implementation

- **Compression**

- The compressor starts by building a frequency table for all byte values (0–255) using NumPy. An additional symbol (256) is added as an **EOF** marker to indicate the end of the stream. From these frequencies, cumulative probability ranges are computed for every symbol.
- The encoder maintains two bounds, **low** and **high**, which initially represent the full coding range. For each input symbol and **EOF** symbol, the current interval is narrowed according to the symbol probability where **range = high - low + 1**.

$$new_low = low + \left\lfloor \frac{range \times cum_low(symbol)}{total} \right\rfloor$$
$$new_high = low + \left\lfloor \frac{range \times cum_high(symbol)}{total} \right\rfloor - 1$$

- After updating the interval, the algorithm applies **renormalization** using three cases:
 - **E1 scaling**: interval lies completely in the lower half.
 - **E2 scaling**: interval lies completely in the upper half.
 - **E3 scaling**: interval lies in the middle half, requiring deferred bits (pending_bits).
- Generated bits are packed into bytes using a buffer before being appended to the output stream.
- Finally, the encoder flushes the remaining interval bits, pads the last byte if necessary, and prepends the frequency table to the compressed output so the decoder can reconstruct the same probability model.

- **Decompression**

- The decompression process begins by reading the **first 1028 bytes** of the compressed file, which contain the frequency table for the 257 symbols (256 byte values + EOF symbol). Using these frequencies, cumulative probability ranges are rebuilt exactly as in the encoder.
- The remaining bytes are converted into a bitstream, and the decoder **initializes a 32-bit code register** using the first bits of the stream.
- For each symbol, the decoder computes a scaled value inside the current interval where **range = high - low + 1**.

$$value = \left\lfloor \frac{((code - low + 1) \times total - 1)}{range} \right\rfloor$$

- The algorithm then finds the symbol whose cumulative range contains this value.
- After identifying the symbol, the interval is narrowed to the corresponding subrange, exactly matching the encoder's operations.
- The decoder continuously applies the same **renormalization** steps used during compression:
 - **E1 scaling**: interval in lower half.
 - **E2 scaling**: interval in upper half.
 - **E3 scaling**: interval in middle half.
- During renormalization, **new bits are shifted into the code register** to keep the decoder synchronized with the encoded bitstream.
- Finally, decoded symbols are appended to the output until all original symbols are reconstructed, **excluding the EOF** marker added during compression.

2. A short explanation of Simplified DEFLATE Implementation:

LZ77 Stage

The simplified DEFLATE pipeline begins with an LZ77 algorithm designed to eliminate redundancy by replacing repeated byte sequences with back-references (Match tokens) and outputting non-repeated bytes directly (Literal tokens).

Token Generation (Compression)

The encoder iterates through the input byte array and attempts to find the longest matching sequence in the previously processed data. To achieve this efficiently without an exhaustive search, the implementation utilizes a dictionary-based hash table. This table groups previous buffer positions using their 3-byte prefixes as keys.

At each position:

- **Match Finding:** The algorithm looks up the current 3-byte prefix in the hash table. It evaluates past occurrences from newest to oldest.
- **Search Constraints:** To ensure optimal performance and adhere to DEFLATE-style limits, the search is restricted to a history window of 32,768 bytes (`WINDOW_SIZE`). Additionally, the search terminates early after evaluating a maximum of 64 candidates (`MAX_CANDIDATES`) to prevent significant slowdowns on highly repetitive data.
- **Match Selection:** The longest valid match is selected, capped at a maximum length of 258 bytes (`MAX_MATCH`). If the longest match is at least 3 bytes (`MIN_MATCH`), a `Match(length, distance)` token is generated. Otherwise, a `Literal(byte)` token is emitted.
- **Dictionary Updates:** As the encoder advances, it continuously inserts the 3-byte sequences of the processed positions into the hash table, ensuring future lookups have an up-to-date history. When a match of length L is found, all L positions covered by the match are hashed and added to the dictionary.

Decompression

The decoder reconstructs the original file by linearly processing the token array into a byte buffer.

- When a `Literal` is encountered, its byte value is appended directly to the buffer.
- When a `Match` is encountered, the decoder calculates the start index by subtracting the match's `distance` from the current buffer length. It then copies the specified `length` of bytes from that position.
- Crucially, the decompression copies data byte-by-byte within a loop. This properly supports overlapping matches (where the match length exceeds the match distance), dynamically copying newly appended bytes as it moves forward.

DEFLATE Symbols & Extra Bits (Intermediate Translation) Stage This stage acts as the bridge between the raw LZ77 tokens and the Canonical Huffman encoder. Because assigning a unique Huffman code to every possible match distance (up to 32,768) would

create an inefficiently large codebook, this stage categorizes exact lengths and distances into standardized DEFLATE symbols and binary "extra bits."

Event Generation (Compression) The encoder iterates through the LZ77 token stream and translates them into DEFLATE-compatible events:

- **Literals:** `Literal` tokens are directly converted into `LiteralEvent` objects using their raw byte value (0–255).
- **Matches:** `Match` tokens are broken down using spec-defined lookup tables (`length_base` and `distance_base`). The exact length and distance are mapped to a specific base category (yielding a length symbol between 257–285, and a distance symbol between 0–29). The remainder is formatted as a binary string of "extra bits" to pinpoint the exact value within that category. The implementation safely handles DEFLATE anomalies, such as strictly mapping a maximum length of 258 to symbol 285 with zero extra bits.
- **Termination:** An `EndEvent(256)` is appended to the stream to signify the end of the block.
- **Frequency Counting:** Before handing the event stream to the Huffman stage, this module pre-calculates the occurrence frequencies of all literal/length symbols (0–285) and distance symbols (0–29), ignoring raw extra bits.

Event Decoding (Decompression) During decompression, this stage receives the decoded event stream from the Huffman module and reconstructs the original file array:

- **Symbol Decoding:** It maps the length and distance symbols back to their base values and adds the integer value of the decoded extra bits to recover the exact LZ77 length and distance instructions.
- **LZ77 Reconstruction:** When a reconstructed Match is encountered, the decoder executes the byte-by-byte copy process (as detailed in Stage 1) to safely build the output buffer.
- **Termination:** The decoder immediately halts reconstruction upon receiving the `EndEvent(256)` symbol, safely ignoring any trailing padding bits.

Canonical huffman & header management

Final step in simplified DEFLATE pipeline where the payload is generated
Compression

1. Take events from Deflate.
2. Build huffman lengths for `literal events` and `distance events` which is the typical huffman algorithm producing free-prefix trees.
3. Build `canonical huffman encoding` which is giving each symbol a unique representation in binary based on its location in the tree.
4. Encode events using the canonical huffman encoding to get `payload`.
5. Get `literal_bitwidth` which is the highest count in literal huffman lengths and write it.

6. Get `distance_bitwidth` which is the highest count in distance huffman lengths and write it.
7. Write each bitwidth for each symbol in literal huffman lengths then distance huffman lengths and write it.
8. Write `payload`.

Decompression

1. read `literal_bitwidth`
2. read `distance_bitwidth`
3. read `literal huffman lengths`
4. read `distance huffman lengths`.
5. read `payload`
6. build reverse encoding for canonical huffman encoding.
7. decode events using reverse encoding
8. huffman decompression is done.

3. The file formats used by both compressors

Range Coding (`.rng`)

Field	Size	Description
<code>FREQ_TABLE</code>	256 × 4 bytes	Raw byte frequencies (big-endian <code>uint32</code>)
<code>PAYLOAD</code>	variable	Encoded interval bytes

Simplified DEFLATE (`.sdf1`)

Field	Size	Description
<code>LIT_BW</code>	4 bits	Bit-width for literal/length code lengths
<code>DIST_BW</code>	4 bits	Bit-width for distance code lengths
<code>LIT_TABLE</code>	286 × <code>LIT_BW</code> bits	Literal/length code lengths
<code>DIST_TABLE</code>	30 × <code>DIST_BW</code> bits	Distance code lengths
<code>PAYLOAD</code>	variable	Huffman-coded data + raw extra bits

4. A table comparing compression results on different files.

File	Original Size	DEFLATE Size	DEFLATE Ratio	Range Size	Range Ratio	Best Method
small text	190 B	~260 B	1.3684	~1,135 B	5.9737	— (both expand)
large text	19,716 B	~4,620 B	0.2342	~11,950 B	0.6060	DEFLATE
repeated	3,559 B	~151 B	0.0424	~2,634 B	0.7398	DEFLATE
source code	2,057 B	~748 B	0.3636	~2,236 B	1.0870	DEFLATE
random binary	2,048 B	~2,176 B	1.0625	~3,051 B	1.4902	— (both expand)

5. A short discussion of the results.

1. Deflate is better in most cases than Range coding. The reason for this is the reduction of repeated patterns that is in LZ77. Which is much better than range coding as it uses a naive cumulative array.
2. Both Deflate and range coding are bad at random files as they depend on reducing patterns in file and giving more attention to higher frequency elements. Therefore, uniformly distributed files are useless against them and need another kind of algorithm.
3. Bad effect on small files is due to the limited size of metadata for each algorithm which will be easily broken as the file gets larger.
4. Deflate is faster in all of the cases above.

Discussion Questions

1. Which method compressed repeated text better?
 - a. Deflate is better in most cases than Range coding. The reason for this is the reduction of repeated patterns that is in LZ77. Which is much better than range coding as it uses a naive cumulative array.

2. Which method compressed normal English text better?

- a. Deflate is the one to go as the english text is not uniformly distributed. Therefore, both will work well. Moreover, Deflate is better due to pattern matching done by LZ77. It is well-known that old cyber attacks that are used in encryption are broken using frequency attacks.

3. What happened with random data?

- a. Both Deflate and range coding are bad at random files as they depend on reducing patterns in file and giving more attention to higher frequency elements. Therefore, uniformly distributed files are useless against them and need another kind of algorithm. As Huffman will make a nearly balanced tree with this kind of data. Moreover, this indicates nearly maximum entropy.

4. Why can small files become larger after compression?

- a. Due to the limited size of metadata for each algorithm which will easily vanish as the file gets larger.

5. Why does DEFLATE usually benefit from LZ77 before Huffman coding?

- a. LZ77 finds repeated byte sequences which makes a huge difference in repeated content in files.

6. Why does range coding need a frequency table?

- a. Range coding needs a frequency table to determine the probability of each symbol. The frequencies are used to divide the coding interval into proportional subranges, where frequent symbols get larger ranges and rare symbols get smaller ranges. Both the encoder and decoder must use the same frequency table so they can map symbols to the exact same ranges during compression and decompression.

7. How does the decoder know when to stop in each method?

- a. In Range Coding, the decoder can stop using either:
 - i. An EOF (End Of File) symbol added during compression.
 - ii. The total number of original symbols obtained from the frequency table. In this implementation, decoding stops after decoding `total - 1` symbols.

and in our implementation we used the second approach.

- b. Deflate: meeting EndEvent symbol.